Complete diagnostic programs for gluing

All three programs are printed in full. They test the written uniform constructions and explicit failed weakenings; none is a premise of the general proofs. The declared program bundle nevertheless includes every line. The printed appendix and the three whole raw files are alternative representations, never added together. Three operational replay drivers and both partial Lean files are inventoried separately.

programs/check_gluing.g

```
# Diagnostic finite models for the written all-subset gluing theorem.
# No finite model list is asserted to prove the theorem for all finite sets.
SetInfoLevel(InfoWarning,0);;
Need:=function(b,s) if not b then Error(s); fi; end;;
OnSubset:=function(marks,S) local x;x:=List(S,i->marks[i]);if IsEmpty(x) then return
↪  Group(());fi;return Group(x);end;;
AllIP:=function(marks) local subs,gs,S,T,i,j,k;subs:=Combinations([1..Length(marks)]);gs:=L
↪  ist(subs,S->OnSubset(marks,S));for i in [1..Length(subs)] do for j in [i..Length(subs)]
↪  do k:=Position(subs,Intersection(subs[i],subs[j]));if Intersection(gs[i],gs[j])<>gs[k]
↪  then return false;fi;od;od;return true;end;;
RootChecks:=function(marks,a) local
↪  subs,gs,orbs,i,j,k,O,K,H,Sym,comp,S,rest;subs:=Combinations([1..Length(marks)]);gs:=Lis
↪  t(subs,S->OnSubset(marks,S));orbs:=List(gs,g->Set(Orbit(g,a)));for i in
↪  [1..Length(subs)] do H:=gs[i];O:=orbs[i];if Length(O)<2 then Sym:=Group(());else
↪  Sym:=SymmetricGroup(O);fi;Need(IsSubgroup(H,Sym),"supported root symmetric group");for j
↪  in [i..Length(subs)] do k:=Position(subs,Intersection(subs[i],subs[j]));Need(Intersecti
↪  on(orbs[i],orbs[j])=orbs[k],"root orbit meet");od;od;return true;end;;
ExtendAtRoot:=function(g,imgs,n) local out,x;out:=[1..n];for x in [1..Length(imgs)] do
↪  out[imgs[x]]:=imgs[x^g];od;return PermList(out);end;;
models:=[
 rec(name:="path",n:=4,marks:=[(1,2),(2,3),(3,4)]),
 rec(name:="branch",n:=4,marks:=[(1,2),(2,3),(2,4)]),
 rec(name:="coupled-tail",n:=5,marks:=[(1,2),(2,3)(4,5)]),
 rec(name:="diagonal-tail",n:=7,marks:=[(1,2),(2,3)(5,6),(3,4)(6,7)]),
 rec(name:="fixed-root",n:=3,marks:=[(2,3)]),
 rec(name:="interior-root",n:=4,marks:=[(1,2),(1,3),(3,4)])
];;
for M in models do Need(AllIP(M.marks),Concatenation("input IP
↪  ",M.name));RootChecks(M.marks,1);od;
pairs:=[[1,1],[1,2],[2,3],[3,3],[3,4],[4,4],[4,5],[5,5],[1,6],[3,6],[4,6],[6,6]];;
for pair in pairs do
 L:=models[pair[1]];R:=models[pair[2]];n:=L.n+R.n-1;
 # Conjugate the second input so point 1 is common and every other point is new.
 imgs:=[1];for i in [2..R.n] do Add(imgs,L.n+i-1);od;
 right:=[];for gg in R.marks do
 ↪  Add(right,ExtendAtRoot(gg,imgs,n));od;joined:=Concatenation(L.marks,right);
 Need(AllIP(joined),"glued full IP");RootChecks(joined,1);
 Print("PAIR ",L.name," / ",R.name," rank ",Length(joined)," degree ",n," order
 ↪  ",Size(Group(joined))," PASS\n");
od;
# Iterated attachment, including a central piece with a coupled remote component.
central:=[(1,2),(1,3),(3,4)(5,6)];;
Need(AllIP(central),"central two-port marking");RootChecks(central,1);RootChecks(central,2);
chain:=Concatenation([(2,7)],central,[(1,8)]);;
Need(AllIP(chain),"two-port iterated gluing");RootChecks(chain,7);RootChecks(chain,8);
Print("ITERATED_CHAIN rank5 degree8 order ",Size(Group(chain))," PASS\n");
branch:=[(1,2),(1,3),(1,4),(2,5),(3,6),(4,7)];;
Need(AllIP(branch),"three-port tree gluing");for p in [5,6,7] do RootChecks(branch,p);od;
Print("ITERATED_TREE rank6 degree7 order ",Size(Group(branch))," PASS\n");
# Literal source counterexample to replacing a pure leaf transposition by a
# label that has an additional, disjoint transposition.
old:=[(1,2)(3,4),(2,3),(3,4)];;bad:=[(1,5),(1,2)(3,4),(2,3),(3,4)];;
Need(AllIP(old),"old source C marking");Need(not AllIP(bad),"impure leaf must fail");
A:=Group(bad{[2,3,4]});;B:=Group(bad{[1,2,3]});;C:=Group(bad{[2,3]});;
Need(Size(A)=24 and Size(B)=120 and Size(C)=8 and Size(Intersection(A,B))=24,"literal
↪  witness 24/120/8");
Print("IMPURE_LEAF_COUNTEREXAMPLE intersection24 common8 PASS\n");
# Two-point overlap cannot replace the one-point hypothesis, even for trees.
left:=[(1,3),(2,3)];;right:=[(1,4),(2,4)];;
```

```
Need(AllIP(left) and AllIP(right),"two trees");Need(not
↪  AllIP(Concatenation(left,right)),"two-point overlap failure");
Need(Size(Intersection(Group(left),Group(right)))=2,"two-point overlap common
↪  transposition");
Print("TWO_POINT_OVERLAP_COUNTEREXAMPLE PASS\n");
# The C-property alone does not force intersections of distinguished orbits.
marks:=[(1,2),(1,2)(3,4)];;
Need(AllIP(marks),"orbit countermodel input C");
Need(Intersection(Set(Orbit(Group(marks{[1]}),1)),Set(Orbit(Group(marks{[2]}),1)))=[1,2],"o
↪  rbit countermodel larger meet");
Need(Size(Intersection(Group(marks{[1]}),Group(marks{[2]})))=1,"orbit countermodel subgroup
↪  meet");
Print("ROOT_SUPPORT_OMISSION_COUNTEREXAMPLE PASS\n");
Need(not AllIP([(1,2),(2,3),(1,3)]),"cyclic attachment failure");
Print("CYCLIC_ATTACHMENT_COUNTEREXAMPLE PASS\n");
Print("PASS 6 inputs, 12 paired plus 2 iterated cases, all subset IP and root-orbit meets, 4
↪  weakening controls\n");
QUIT;
```

programs/check_parallel.g

```
# Complete diagnostics for the written uniform theorems; no finite-premise claim.
SetInfoLevel(InfoWarning,0);;
Need:=function(b,s) if not b then Error(s); fi; end;;
OnSubset:=function(marks,S) local x;x:=List(S,i->marks[i]);if IsEmpty(x) then return
↪  Group(());fi;return Group(x);end;;
AllIP:=function(marks) local subs,gs,i,j,k;subs:=Combinations([1..Length(marks)]);gs:=List(
↪  subs,S->OnSubset(marks,S));for i in [1..Length(subs)] do for j in [i..Length(subs)] do
↪  k:=Position(subs,Intersection(subs[i],subs[j]));if Intersection(gs[i],gs[j])<>gs[k] then
↪  return false;fi;od;od;return true;end;;
Components:=function(n,edges) local cs,e,a,b;cs:=List([1..n],x->[x]);for e in edges do
↪  a:=PositionProperty(cs,c->e[1] in c);b:=PositionProperty(cs,c->e[2] in c);if a<>b then
↪  cs[a]:=Union(cs[a],cs[b]);Remove(cs,b);fi;od;return cs;end;;
Marks:=function(n,edges,Removed) local out,e,g;out:=[];for e in edges do g:=(e[1],e[2]);if
↪  Intersection(e,Removed)=[] then g:=g*(n+e[1],n+e[2]);fi;Add(out,g);od;return out;end;;
SymSupport:=function(S) if Length(S)<2 then return Group(());fi;return
↪  SymmetricGroup(S);end;;
Predicted:=function(n,edges,Removed) local cs,C,gs,D,ed,other,I,x,g;
 cs:=Components(n,edges);gs:=[];
 for C in cs do
  if Intersection(C,Removed)=[] then
   for g in GeneratorsOfGroup(SymSupport(C)) do
   ↪  Add(gs,g*PermList(Concatenation([1..n],List([1..n],x->n+x^g))));od;
  else
   Append(gs,GeneratorsOfGroup(SymSupport(C)));
   ed:=Filtered(edges,e->IsSubset(C,e) and Intersection(e,Removed)=[]);
   for D in Components(n,ed) do if IsSubset(Difference(C,Removed),D) then
   ↪  Append(gs,GeneratorsOfGroup(SymSupport(List(D,x->x+n))));fi;od;
  fi;
 od;
 if IsEmpty(gs) then return Group(());fi;return Group(gs);
end;;
trees:=[rec(n:=1,edges:=[]),rec(n:=2,edges:=[[1,2]]),rec(n:=4,edges:=[[1,2],[2,3],[3,4]]),r
↪  ec(n:=4,edges:=[[1,2],[1,3],[1,4]]),rec(n:=5,edges:=[[1,2],[2,3],[3,4],[4,5]]),rec(n:=5
↪  ,edges:=[[1,2],[2,3],[2,4],[4,5]])];;
cases:=0;;fibers:=0;;wreaths:=0;;
for T in trees do
 for Removed in Combinations([1..T.n]) do
  if Removed<>[] then
   ZE:=Filtered(T.edges,e->IsSubset(Removed,e));
   comps:=Components(T.n,ZE);
   if Removed in comps then
    m:=Marks(T.n,T.edges,Removed);Need(AllIP(m),"deleted tree C");
    for I in Combinations([1..Length(T.edges)]) do
     Need(OnSubset(m,I)=Predicted(T.n,T.edges{I},Removed),"exact selected subgroup
     ↪  formula");fibers:=fibers+1;
    od;
    cases:=cases+1;
   fi;
  fi;
 od;
 for z in [1..T.n] do
```

```
  m:=Marks(T.n,T.edges,[z]);s:=Product([1..T.n],i->(i,T.n+i));
  Need(AllIP(Concatenation(m,[s])),"tree wreath C");
  Need(Size(Group(Concatenation(m,[s])))=2*Factorial(T.n)^2,"full wreath order");
  wreaths:=wreaths+1;
 od;
od;
# Disconnected deleted set is the source's p9 unequal-start counterexample.
bad:=Marks(4,[[1,2],[2,3],[3,4]],[1,4]);;
Need(not AllIP(bad),"disconnected deletion must fail");
Need((6,7) in Intersection(Group(bad{[1,2]}),Group(bad{[2,3]})) and not (6,7) in
↪  Group(bad{[2]}),"literal defect pure second transposition");
# A cycle instead of a tree already fails in one row.
Need(not AllIP(Marks(3,[[1,2],[2,3],[1,3]],[1,2,3])),"cycle must fail");
# Exact marked identification, not just abstract group order, for the source r3 row.
source:=[(1,2)(5,6),(2,3),(1,5)(2,6)(3,4)];;
Need(AllIP(source) and Size(Group(source))=72,"literal source rank3 marking");
Print("PASS deleted cases ",cases," exact selected subgroups ",fibers," wreath cases
↪  ",wreaths," 2 weakening controls and literal rank3 source model\n");


# The individual-flip theorem is tested on all roots of the same trees.
binaries:=0;; binaryfibers:=0;;
for T in trees do
 for z in [1..T.n] do
  m:=Marks(T.n,T.edges,[]);f:=(z,T.n+z);
  Need(AllIP(Concatenation(m,[f])),"binary paired-tree C");
  Need(Size(Group(Concatenation(m,[f])))=2^T.n*Factorial(T.n),"binary full order");
  for I in Combinations([1..Length(T.edges)]) do
   cs:=Components(T.n,T.edges{I});D:=First(cs,C->z in C);
   predicted:=Group(Concatenation(m{I},List(D,x->(x,T.n+x))));
   Need(OnSubset(Concatenation(m,[f]),Concatenation(I,[Length(m)+1]))=predicted,"binary
   ↪  exact fiber");
   binaryfibers:=binaryfibers+1;
  od;
  binaries:=binaries+1;
 od;
od;
# Every selected group containing both new marks has the stated product formula.
tailfibers:=0;;
for n in [1..5] do
 edges:=List([1..n-1],i->[i,i+1]);m:=Marks(n,edges,[n]);s:=Product([1..n],i->(i,n+i));t:=(2⌋
 ↪  *n,2*n+1);
 marks:=Concatenation(m,[s,t]);Need(AllIP(marks),"terminal-tail
 ↪  C");Need(Size(Group(marks))=Factorial(2*n+1),"terminal-tail full group");
 for I in Combinations([1..Length(edges)]) do
  cs:=Components(n,edges{I});D:=First(cs,C->n in
  ↪  C);R:=Concatenation(D,List(D,x->n+x),[2*n+1]);
  gs:=ShallowCopy(GeneratorsOfGroup(SymSupport(R)));
  for C in cs do if C<>D then
   for g in GeneratorsOfGroup(SymSupport(C)) do
   ↪  Add(gs,g*PermList(Concatenation([1..n],List([1..n],x->n+x^g))));od;
  fi;od;
  if Difference([1..n],D)<>[] then Add(gs,Product(Difference([1..n],D),x->(x,n+x)));fi;
  Need(Group(gs)=OnSubset(marks,Concatenation(I,[n,n+1])),"tail exact double-mark
  ↪  formula");tailfibers:=tailfibers+1;
 od;
od;
# Rooted-tree tail weakening: explicit element, not merely a failed black-box IP test.
m:=[(1,2),(1,3),(1,4)(2,5)(3,6),(4,7)];;
Need((3,6) in Group(m{[1,3,4]}) and (3,6) in Group(m{[2,3,4]}) and not (3,6) in
↪  Group(m{[3,4]}),"branching tail obstruction");
Print("PASS binary cases ",binaries," exact binary fibers ",binaryfibers," exact tail fibers
↪  ",tailfibers," and branching-tail control\n");
QUIT;
```

programs/check_applications.g

```
# Complete diagnostics for the written uniform theorems; no finite-premise claim.
SetInfoLevel(InfoWarning,0);;
Need:=function(b,s) if not b then Error(s); fi; end;;
OnSubset:=function(marks,S) local x;x:=List(S,i->marks[i]);if IsEmpty(x) then return
↪  Group(());fi;return Group(x);end;;
```

```
AllIP:=function(marks) local subs,gs,i,j,k;subs:=Combinations([1..Length(marks)]);gs:=List(
↪  subs,S->OnSubset(marks,S));for i in [1..Length(subs)] do for j in [i..Length(subs)] do
↪  k:=Position(subs,Intersection(subs[i],subs[j]));if Intersection(gs[i],gs[j])<>gs[k] then
↪  return false;fi;od;od;return true;end;;
Components:=function(n,edges) local cs,e,a,b;cs:=List([1..n],x->[x]);for e in edges do
↪  a:=PositionProperty(cs,c->e[1] in c);b:=PositionProperty(cs,c->e[2] in c);if a<>b then
↪  cs[a]:=Union(cs[a],cs[b]);Remove(cs,b);fi;od;return cs;end;;
Marks:=function(n,edges,Removed) local out,e,g;out:=[];for e in edges do g:=(e[1],e[2]);if
↪  Intersection(e,Removed)=[] then g:=g*(n+e[1],n+e[2]);fi;Add(out,g);od;return out;end;;
SymSupport:=function(S) if Length(S)<2 then return Group(());fi;return
↪  SymmetricGroup(S);end;;
Predicted:=function(n,edges,Removed) local cs,C,gs,D,ed,other,I,x,g;
 cs:=Components(n,edges);gs:=[];
 for C in cs do
  if Intersection(C,Removed)=[] then
   for g in GeneratorsOfGroup(SymSupport(C)) do
   ↪  Add(gs,g*PermList(Concatenation([1..n],List([1..n],x->n+x^g))));od;
  else
   Append(gs,GeneratorsOfGroup(SymSupport(C)));
   ed:=Filtered(edges,e->IsSubset(C,e) and Intersection(e,Removed)=[]);
   for D in Components(n,ed) do if IsSubset(Difference(C,Removed),D) then
   ↪  Append(gs,GeneratorsOfGroup(SymSupport(List(D,x->x+n))));fi;od;
  fi;
 od;
 if IsEmpty(gs) then return Group(());fi;return Group(gs);
end;;

StringCheck:=function(m) local i,j;for i in [1..Length(m)] do for j in [i+2..Length(m)] do
↪  if m[i]*m[j]<>m[j]*m[i] then return false;fi;od;od;return true;end;;
folded:=0;; auxiliary:=0;; negative:=0;;
for n in [2..5] do
 m:=Concatenation([(1,n+1)],List([1..n-1],i->(i,i+1)(n+i,n+i+1)),[(n,2*n+1)]);
 Need(StringCheck(m) and AllIP(m) and Size(Group(m))=Factorial(2*n+1),"folded tail
 ↪  theorem");folded:=folded+1;
od;
for r in [2..5] do
 m:=List([1..r-2],i->(i,i+1)(r+2+i,r+3+i));
 Add(m,(r-1,r)(2*r+1,2*r+2)(r+1,r+2));Add(m,(r,r+1));
 Need(StringCheck(m) and AllIP(m),"auxiliary family C");
 if r=2 then expected:=8;else expected:=Factorial(r+2)*Factorial(r);fi;
 Need(Size(Group(m))=expected,"auxiliary full group");auxiliary:=auxiliary+1;
od;
c:=0;;len:=0;;x:=[];;y:=[];;
for h in [1..3] do for r in [h+4..h+6] do
 n:=h+1;len:=r-h-2;c:=2*n+1;x:=List([1..len],j->c+j);y:=List([1..len],j->c+len+j);
 f:=(1,n+1);d:=List([1..h],j->(j,j+1)(n+j,n+j+1));t:=(n,c);tau:=Product([1..len],j->(x[j],y
 ↪  [j]));ss:=t*tau;u:=(c,x[1]);
 e:=List([1..len-1],j->(x[j],x[j+1])(y[j],y[j+1]));
 m:=Concatenation([f],d,[ss,u],e);P:=Group(Concatenation([f],d,[ss]));Q:=Group(Concatenatio
 ↪  n(d,[ss,u]));K:=Group(Concatenation(d,[ss]));
 Need(tau in P and tau in Q and not tau in K,"literal parity defect");
 Need(StringCheck(m) and Size(Group(m))=Factorial(2*r-1),"negative family full symmetric
 ↪  string group");
 negative:=negative+1;
od;od;
Print("PASS folded ",folded," auxiliary ",auxiliary," uniform negative ",negative," cases;
↪  explicit parity defects checked\n");

# Source's second impure-leaf example: diagnostic for the written short proof.
a:=(1,2)(5,6);b:=(2,3)(4,5)(6,7);c:=(3,4);x:=(8,1);;
Need(AllIP([a,b,c]) and Size(Group(a,b,c))=5040,"second source input marking");
Need(Size(Group(b,c))=8 and
↪  Set(Orbit(Group(b,c),[2,3],OnSets))=Set([[2,3],[2,4],[3,5],[4,5]]),"second source
↪  orbit");
Need(Intersection(Group(a,b),Group(b,c))=Group(b),"second source end intersection");
Need(Intersection(Group(a,b,c),Group(x,a))=Group((1,2),(5,6)) and not (5,6) in
↪  Group(a),"second source literal failure");
Print("PASS second source impure-leaf example, exact orbit and intersection\n");
QUIT;
```